The SMTLIB Plugin

1.0 Introduction....1
1.1 Commands....1
1.1.1 smt....1
1.1.2 smtexp....2
1.2 Design....3
1.2.1 Analysis Plugins....3
1.2.2 Analysis Commands....3
1.2.3 SMTLIB....3
1.2.4 Translation Visitors....4
1.2.5 Solvers....5
1.3 Issues....6
1.4 Future Directions....6
1.5 Source....7
1.6 References....7

1.0 Introduction

The SMTLIB plugin is experimental. It enables some VDMJ [1] Proof Obligations to be translated into the SMT-LIB [4] language and be submitted to an SMT solver [7]. The outcome of the solver will be interpreted as one of:

- PROVED: the obligation is true in all cases
- FAILED: a counterexample has been found by the solver
- MAYBE: neither of the above.

Note that these are similar to QuickCheck [3] outcomes. However obligations may be *proved* by this tool, rather than merely indicating that they are *probably provable*. This is because an SMT solver is considering all possible values, by symbolic reasoning, unlike QuickCheck.

The tool is experimental. It is currently only capable of processing simple obligations that concern basic types (numerics and booleans, and sets/seqs/maps of these). But the hope is that the architecture of the plugin is sound, which will simplify further development over time.

1.1 Commands

The plugin offers two commands, *smt* and *smtexp*.

1.1.1 smt

The *smt* command is available in both the VDMJ command line and the VDM-VSCode [2] Console.

```
> help smt
smt [-v][-<solver>] <PO#> [<PO#>...] – experimental SMT solver PO verification
>
```

The -v option (verbose) lists the SMT-LIB source that is generated for each PO considered. The second argument is the solver to use. By default, the command will use the Z3 solver [5], but an adapter for CVC5 [6] is also available. So the option -cvc5 would use that.

```
> pog
Generated 1 proof obligation:
```

```
Proof Obligation 1: (Unproved)
f: subtype obligation in 'DEFAULT' (test.vdm) at line 2:5
(forall a:nat &
  (a - 1) >= 0)

> smt 1
PO #1, FAILED in 0.005s
f: subtype obligation in 'DEFAULT' (test.vdm) at line 2:5
(forall a:nat &
  (a - 1) >= 0)
---- Counterexample:
a = 0
>
```

The tool is designed to allow arbitrary solvers to be added in future.

Note that the chosen solver must be installed on the machine separately, such that it can be executed on the default path with the name “z3” or “cvc5”. If the solver chosen is not installed, or there is no adapter for it, you will get the following error:

```
> smt -z4 1
Failed to execute solver: Unknown solver type: z4
>
```

1.1.2 smtexp

The *smtexp* command is only available on the VDMJ command line and is intended for debugging. Rather than trying to solve proof obligations, the command takes an arbitrary VDM boolean expression and attempts to solve that as though it was the body of a PO. This is much easier than trying to write a VDM spec that has obligations that contain an expression that you want to test. The command constructs a PO #0 containing the expression and then solves that.

```
> smtexp
Usage: smtexp [-<solver>] <expression>
>
```

The command is roughly the same as generating a proof obligation with the expression passed, and then running the *smt* command with the -v verbose option. That is, it always displays the SMT-LIB output as well as the solver result.

```
> smtexp -z3 forall s:seq of nat, i:nat & s(i) = 0
; Proof Obligation #0
(set-option :produce-models true)
(set-info :smt-lib-version 2.6)
(set-logic ALL)
; Using solver z3
(push)
...
(check-sat)
(get-model)
(pop)
----
PO #0, FAILED in 0.023s
smtexp: theorem obligation
forall s:seq of nat, i:nat & s(i) = 0
---- Counterexample:
s = [2]
i = 0
>
```

1.2 Design

1.2.1 Analysis Plugins

Plugins are provided for the VDMJ command line and for the VDM-VSCode Console. These in turn contribute analysis commands which offer the *smt* and *smtexp* functionality. Both the VDMJ and VDM-VSCode commands call a common implementation class that does the work. This pattern keeps the different plugins and their commands separate from the main implementation.

Figure 1 shows the overall architecture. The orange parts are VDMJ components, and the blue parts are VDM-VSCode components. Both call the common grey components.

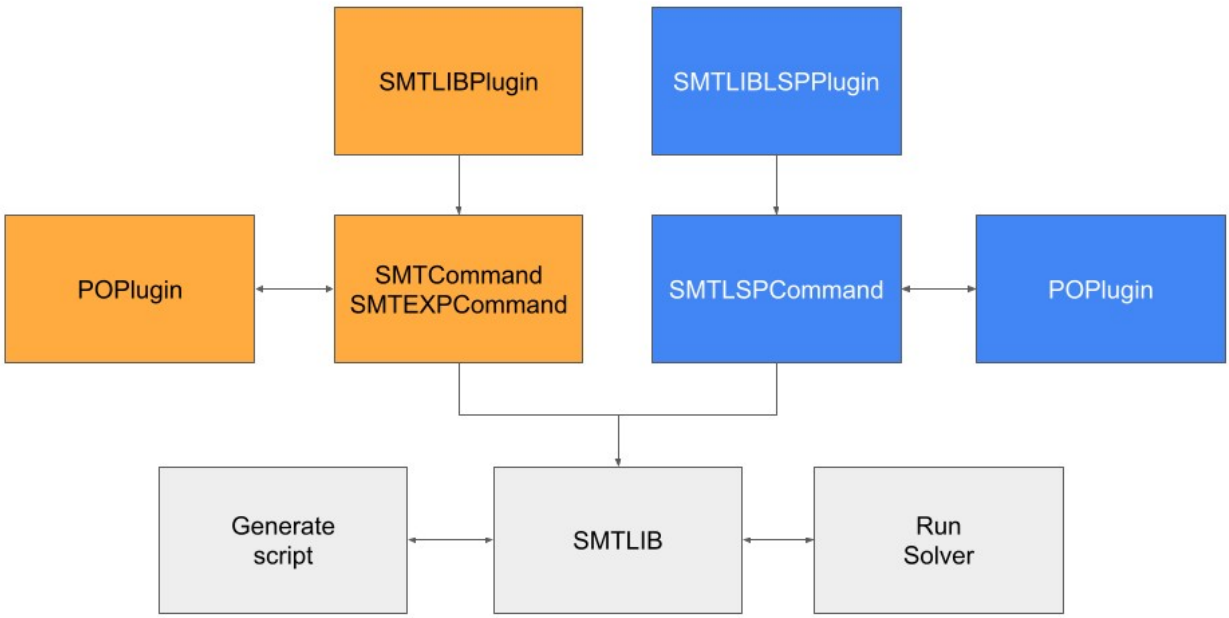


Figure 1: VDMJ and VDM-VSCode plugin and command structure

1.2.2 Analysis Commands

The analysis commands are responsible for parsing command line options, finding the proof obligations concerned, generating an SMT-LIB script that represents each obligation (or the expression passed to *smtexp*), invoking the chosen solver and displaying the result on the console.

Proof obligations are obtained from the *POPlugin.* The generation of SMT-LIB scripts and the execution of the solver is performed via the common SMTLIB class.

See Figure 1.

1.2.3 SMTLIB

The common SMTLIB class contains all of the functionality to generate SMT-LIB scripts and invoke a solver to analyse them.

The class is constructed with Environment and Solver objects. The Environment allows the lookup of definitions within the current specification, which is needed during SMT-LIB generation. The

Solver object enables interaction with one particular solver. Implementations are provided for the Z3 and CVC5 solvers, but others can be added.

There are two public methods:

- ***generate***. This is passed a proof obligation and produces a Script object, which is a sequence of SMT-LIB Commands (not to be confused with plugin commands). The bulk of the script is provided by the private *topLevelDefinitions* method.
- ***runSolver***. This is passed a Script and the proof obligation. It uses the Solver to create a child process and pass the script, then interprets the solver response and updates the PO status. This uses a private *parseModel* method (below).

There are two private methods:

- ***topLevelDefinitions***: This is passed a proof obligation and returns a Script, comprising SMT-LIB definitions of the PO expression and any additional functions that it requires (eg. if a precondition is called, it will be converted to a define-func). The method looks for the top level *forall* or *exists* expression and extracts its variable bindings, converting these into top level undefined values. The body of the expression is then translated using the visitors described below. In addition, the method uses a visitor to create additional top level definitions that are needed because the PO expression uses VDM operators which are not directly expressible in SMT-LIB.
- ***parseModel*:** This is passed a proof obligation and an open BufferedReader which is ready to read a counterexample (ie. following a “sat” response from the solver). The model is actually parsed using the Solver, since different solvers produce subtly different model formats. The *parseModel* updates the status, counterexample/witness and “explanation” of the PO.

1.2.4 Translation Visitors

The bulk of the translation from VDMJ Expressions into SMT-LIB Expressions is performed by visitors which analyse the PO expression tree. There are three important visitors:

- ***SMTExpressionAnalysis***: This analyses a PO Expression, returning a Script which contains top level definitions to include before the translation of the PO itself. For example:
  - Functions called by the PO have to be translated, including any functions that they call recursively.
  - Local definitions which represent VDM value constants are translated into SMT-LIB definitions if they are used.
  - Expressions which can produce undefined VDM values, such as *cases* without *others* clauses, add to a singleton called *Globals*. These also produce top level definitions, but the class caters for duplicates.
  - Expressions which have no SMT-LIB equivalent, such as VDM’s “inds” of a sequence, generate top level definitions of functions that provide these features. Sometimes these are explicit “define-func” definitions; sometimes they are implicit “declare-func” definitions with associated assert clauses to define their semantics.
- ***TypeToSMT***: This converts VDM types into SMT-LIB Sorts. Note that:
  - Although SMT-LIB defines primitive types, like Bool, Real and Int, it does not understand nat, nat1 or rat. This means that these VDM types have to be translated into both the raw SMT-LIB type, plus a qualifier that says which values of that type are

legitimate. For example, a “nat1” would be converted to an “Int”, but with the qualifier that the value must be >= 1. A Sort plus a qualifier is held in a *QualifiedSort*.

- The handling of these qualifiers can become quite complicated, when we are trying to qualify as few instances as possible, especially when qualifications percolate up from the leaves of the expression as well as down from the bindings that created them.
- Note that there are two distinct types of constraint. SMT-LIB sorts that are constrained to be VDM types are “given” constraints, whereas constraints that come from VDM partial operators (like *k in set dom m*) are “required”. So we can depend on the given constraints, but if we violate the required constraints, that’s a counterexample.
- Only simple types are catered for currently, like bool, numerics and set/seq/maps of these.

- ***ExpressionToSMT***: This is the main visitor that translates a VDMJ Expression into an SMT-LIB Expression. For the most part, if SMT-LIB has an equivalent operator, the translation is simple. A hierarchy of classes in the “ast” package can represent SMT-LIB expressions, reflecting the grammar defined in the Standard [4]. However, note:
  - Solvers have different levels of compliance with the SMT-LIB standard and some have non-standard (but very useful) extensions for some theories (like sets, seqs and maps). Therefore the translation process has to include the Solver object to take advantage of its capabilities.

1.2.5 Solvers

A number of SMT solvers are potentially available for this tool to use [7]. They all have different strengths and weaknesses, as well as varying compliance with the SMT-LIB Standard.

We want to be able to choose different solvers in the tool, so rather than hard coding support for particular ones, they are represented internally by an abstract Solver class, which gives the tool a standardised interface to work with. Solver instances are then created at startup (and cached) by looking for a Java resource called “smt.solvers”. This contains a list of Solver subclasses to use.

Solvers have the following methods:

- ***getInstance:*** This static method returns the current instance of a given name, like “z3”. An overloaded method has no name and returns the default Solver (currently z3). So this implements a partial-singleton pattern, with one instance per name.
- ***getName:*** This method returns the simple name of the solver, like “z3” or “cvc5”. This is used to process command arguments that select a solver type.
- ***getProcess***: This returns a Java ProcessBuilder which will start the solver instance in such a way that it will accept an SMT-LIB script via its standard input. The process should include any optimizing flags that are required.
- ***getModelNameValue***: This method is passed a counterexample binding from the solver and returns a VDM value string and a name. For example, a solver might return a model value in the form “(define-fun a () Real (/ 1 2))”. That would be converted into “a = 1/2” in VDM terms, so that it can sensibly be displayed as a counterexample. This translation is solver-specific.
- ***getExplanation:*** This method returns an “explanation” for a PO, which includes the PO source as well as the name/value bindings of a witness or counterexample that has been found.

In addition to these, a Solver will implement an interface called *VDMSolver*, which defines a set of methods for translating VDM operators whose equivalent is different in different solvers. These are then called by *ExpressionToSMT* rather than directly encoding the translation.

The “smt.solvers” resource is included with the plugin jar and contains the following entries:

```
smtlib.solvers.Z3Solver
smtlib.solvers.CVC5Solver
```

These are the two concrete subclasses of Solver which implement the Z3 and CVC5 adapters. But if you want to provide a further solver, package it into its own jar, add an “smt.solvers” resource listing the new class, and add the jar to the classpath. The tool’s initialization will instantiate the new solver as well as the standard ones, and you will be able to specify (say) “*smt -opensmt ...*”

1.3 Issues

In no particular order:

- The current implementation only handles very simple PO expressions, which implies very simple VDM specifications. Types are limited, as are patterns. If you attempt to process something it can’t handle, it will fail with a very basic explanation.
- It is surprisingly easy to defeat an SMT solver with even a modestly complex PO which (to a human) is clearly false. They often come back as “unk”, which we treat as MAYBE. It is possible to overwhelm a solver simply with the constraints for VDM types. For example, a PO reasoning about a “seq of nat” has to explicitly say that every element of the “Int” sequences considered is >=0. But that alone was enough to defeat CVC5; changing the expression to “seq of int” (ie. with no constraints) worked fine.
- The Z3 solver is much better at sets and seqs than CVC5, but it is not as good with maps. Z3 is the default for this reason.
- It is annoying that we have to use the very latest versions of the solvers, because although these are the most powerful, they also have the most non-standard features. That means that the Solver classes have to do more work.
- Solvers do typically have a lot of command line options for fine-tuning their behaviour. It may be that some of these would be useful for us. I did try a few at random, but there was no clear change in performance.
- There was an early high-level decision to use SMT-LIB source as the means to communicate with the solvers. It is widely supported and supposedly common. API bindings do exist for other languages in some solvers, but the standardization here is even looser than SMT-LIB.

1.4 Future Directions

Again, in no particular order:

- We should look at whether other language bindings are more powerful than SMT-LIB. We can consider simple “API” bindings, which effectively allow you to do what SMT-LIB does but via Java or Python etc. But we can also consider more radical translations, such as to Dafny or Boogie, and let them worry about the solvers, while we worry about how to interpret their output in VDM terms.
- But we should also be careful about the compromise between flexibility (to use multiple solvers) and complexity (that this introduces in the translation process and the tool environment and its maintenance).

- We can potentially analyse a PO expression and choose a solver based on what features are needed and which solvers are best at that. Similarly, multiple solvers could be invoked if the weaker ones return “unk” rather than a clear “sat” or “unsat”.
- Translating more complex VDM specifications is undoubtedly possible, but if solvers are easily choked by comparatively simple problems, this may not be a useful investment of time.
- On the other hand, from a teaching perspective, very simple specifications are quite normal and using SMT to genuinely prove POs for students might be a useful feature.
- It would be wise to keep an eye on the latest SMT solver releases and features, since their capabilities already vary considerably and new features are coming out all the time.
- One particularly interesting possibility is the combination of an SMT solver with the Lean environment and/or translation. See https://github.com/ufmg-smite/lean-smt.

1.5 Source

The source code for the SMTLIB plugin is currently in a private repo, though it will be released under an open source licence (with VDMJ) when it’s considered stable/useful.

If you would like early access, contact the VDMJ maintainer: nick.battle@gmail.com

1.6 References

[1] VDMJ, https://github.com/nickbattle/vdmj

[2] VDM-VSCode, https://github.com/overturetool/vdm-vscode

[3] QuickCheck, https://github.com/nickbattle/vdmj/wiki/Using-QuickCheck

[4] The SMT-LIB Standard: Version 2.6, http://www.SMT-LIB.org

[5] The Z3 theorem prover, https://github.com/z3prover/z3

[6] The CVC5 theorem prover, https://github.com/cvc5/cvc5

[7] SMT solvers, https://en.wikipedia.org/wiki/Satisfiability_modulo_theories